CHAPTER

21 SOFTWARE TESTING—SPECIALIZED TESTING FOR MOBILITY

The same sense of urgency that drives MobileApp projects also pervades all mobility projects. Stakeholders are worried that they will miss a market window and press to introduce the MobileApp to its intended market. Technical activities that often occur late in the process, such as performance and security testing, are sometimes given short shrift. Usability testing that should occur during the design phase may end up being deferred until just before delivery. These can be catastrophic mistakes. To avoid this situation, you and other team members must ensure that each work product exhibits high quality, or users will move to a competing product [Soa11].

KEY CONCEPTS

accessibility testing 432
alpha test 430
beta test 430
compatibility testing 414
content testing 420
documentation testing 434
internationalization 423
load testing 425
model-based testing 429
navigation testing 421
performance testing 424
real-time testing 426
recovery testing 417
security testing 423
stress testing 425
testing AI systems 428
testing guidelines 413
test strategies for MobileApps 413
test strategies for WebApps 418
usability testing 430

QUICK LOOK

What is it? Mobility testing is a collection of related activities with a single goal: to uncover errors in MobileApp content, function, usability, navigability, performance, capacity, and security.

Who does it? Software engineers and other project stakeholders (managers, customers, end users) all participate in mobility testing.

Why is it important? If end users encounter errors or difficulties within the MobileApp, they will go elsewhere for the personalized content and function they need.

What are the steps? The mobility testing process begins by focusing on user-visible aspects of the MobileApp and proceeds to tests that exercise technology and infrastructure.

What is the work product? A MobileApp test plan is often produced. A suite of test cases is developed for each testing step, and an archive of test results is maintained for future use.

How do I ensure that I've done it right? Although you can never be sure that you've performed every test that is needed, you can be certain that testing has uncovered errors (and that those errors have been corrected). In addition, if you've established a test plan, you can check to ensure that all planned tests have been conducted.

MobileApp requirements and design models cannot be tested solely with executable test cases. You and your team should conduct technical reviews (Chapter 16) that examine usability (Chapter 12) as well as MobileApp performance and security.

There are several important questions to ask when creating a mobility testing strategy [Sch09]:

- Do you have to build a fully functional prototype before you test with users?
- Should you test with the user's device or provide a device for testing?
- What devices and user groups should you include in testing?
- What are the trade-offs associated with lab testing versus remote testing?

We address each of these questions throughout this chapter.

21.1 Mobile Testing Guidelines

MobileApps that run entirely on a mobile device can be tested using traditional software testing methods (Chapters 19 and 20). Alternatively, they can be tested using emulators running on personal computers. Things become more complicated when thin-client MobileApps[1] are to be tested. They exhibit many of the same testing challenges found in WebApps (Section 20.2), but thin-client MobileApps have the additional concerns associated with transmission of data through Internet gateways and telephone networks [Was10].

In general, users expect MobileApps to be context aware and deliver personalized user experiences based on the physical location of a device in relation to available network features. Testing MobileApps in dynamic ad hoc networks for every possible device and network configuration is difficult, if not impossible.

MobileApps are expected to deliver much of the complex functionality and reliability found in desktop applications, but they are resident on mobile platforms with relatively limited resources. The following guidelines provide a basis for mobile application testing [Kea07]:

- Understand the network and device landscape before testing to identify bottlenecks (Section 21.6).
- Conduct tests in uncontrolled real-world test conditions (field-based testing, Section 21.8).
- Select the right automation test tool (Section 21.11).
- Use the Weighted Device Platform Matrix method to identify the most critical hardware/platform combination to test (Section 21.8).
- Check the end-to-end functional flow in all possible platforms at least once (Section 21.10).

1 Thin-client apps typically have software for the user interface running on the mobile device (or Web browser software) and use a network interface to an Internet-based application or cloud-based data storage.

- Conduct performance testing, GUI testing, and compatibility testing using actual devices (Sections 21.8 and 21.11).
- Measure performance only in realistic conditions of wireless traffic and user load (Section 21.8).

21.2 The Testing Strategies

The strategy for testing mobile applications adopts the basic principles for all software testing. However, the unique nature of MobileApps demands the consideration of a number of specialized issues:

- **User-experience testing.** Users are involved early in the development process to ensure that the MobileApp lives up to the usability and accessibility expectations of the stakeholders on all supported devices (Section 21.3).
- **Device compatibility testing.** Testers verify that the MobileApp works correctly on all required hardware and software combinations (Section 21.9).
- **Performance testing.** Testers check nonfunctional requirements unique to mobile devices (e.g., download times, processor speed, storage capacity, power availability) (Section 21.8).
- **Connectivity testing.** Testers ensure that the MobileApp can access any needed networks or Web services and can tolerate weak or interrupted network access (Section 21.6).
- **Security testing.** Testers ensure that the MobileApp does not compromise the privacy or security requirements of its users (Section 21.7).
- **Testing in the wild.** The app is tested under realistic conditions on actual user devices in a variety of networking environments around the globe (Section 21.9).
- **Certification testing.** Testers ensure that the MobileApp meets the standards established by the app stores that will distribute it.

Technology alone is not sufficient to guarantee commercial success of a MobileApp. Users abandon MobileApps quickly if they do not work well or fail to meet expectations. It is important to recall that testing has two important goals: (1) to create test cases that uncover defects early in the development cycle and (2) to verify the presence of important quality attributes. The quality attributes for MobileApps are based on those set forth in ISO 2050:2011 [ISO17] and encompass functionality, reliability, usability, efficiency, maintainability, and portability (Chapter 17).

Developing a MobileApp testing strategy requires an understanding of both software testing and the challenges that make mobile devices and their network infrastructure unique [Kho12a]. In addition to a thorough knowledge of conventional software testing approaches (Chapters 19 and 20), a MobileApp tester should have a good understanding of telecommunications principles and an awareness of the differences and capabilities of mobile operating systems platforms. This basic knowledge must be complemented with a thorough understanding of the different types of mobile testing (e.g., MobileApp testing, mobile handset testing, mobile website testing), the use

of simulators, test automation tools, and remote data access services (RDA). Each of these topics is discussed later in this chapter.

21.3 User Experience Testing Issues

In a crowded marketplace in which products provide the same functionality, users will choose the MobileApp that is easiest to use. The user interface and its interaction mechanisms are visible to the MobileApp users. It is important to test the quality of the user experience provided by the MobileApp to ensure that it meets the expectations of its users.

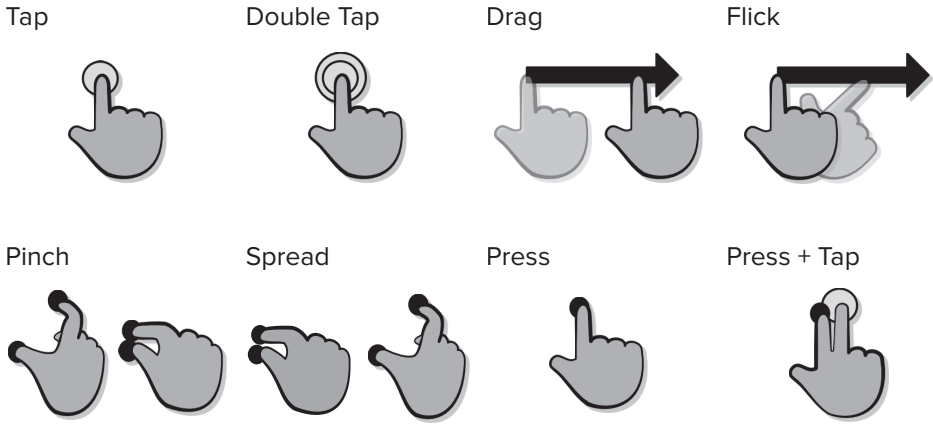
What characteristics of MobileApp usability become the focus of testing, and what specific objectives are addressed? Many of the procedures for assessing the usability of software user interfaces discussed in Chapters 12 and 13 can be used to assess MobileApps. Similarly, many of the strategies used to assess the quality of WebApps (Section 21.5) may be used to test the user interface portion of the MobileApp. There is more to building a good MobileApp user interface than simply shrinking the size of a user interface from an existing desktop application.

21.3.1 Gesture Testing

Touch screens are ubiquitous on mobile devices, and, as a consequence, developers have added multitouch gestures (e.g., swiping, zooming, scrolling, selection) as a means of augmenting the user interaction possibilities without losing screen real estate. Figure 21.1 shows several commonly found gestures on MobileApps. Unfortunately, gesture-intensive interfaces present a number of review and testing challenges.

Paper prototypes, sometimes developed as part of the design, cannot be used to adequately review the adequacy or efficacy of gestures. When testing is initiated, it's difficult to use automated tools to test touch or gesture interface actions. The location

FIGURE 21.1
Mobile app gestures



of screen objects is affected by screen size and resolution, as well as previous user actions, making accurate gesture testing difficult. And even as testing is conducted, gestures are hard to log accurately for replay.

Instead, testers need to create test framework programs that make calls to functions that simulate gesture events. All of this is expensive and time consuming.

Accessibility testing for visually impaired users is challenging because gesture interfaces typically do not provide either tactile or auditory feedback. Usability and accessibility testing for gestures become very important for ubiquitous devices like smartphones. It may be important to test the operation of the device when gesture operations are not available.

Ideally, user stories or use cases are written in sufficient detail to allow their use as the basis for test scripts. It is important to recruit representative users and include all targeted devices to take screen differences into account when testing gestures with a MobileApp. Finally, testers should ensure that the gestures conform to the standards and contexts set for the mobile device or platform.

21.3.2 Virtual Keyboard Input

Because a virtual keyboard may obscure part of the display screen when activated, it is important to test the MobileApp to ensure that important screen information is not hidden from the user while typing. If the screen information must be hidden, it is important to test the ability of the MobileApp to allow page flipping by the user without losing typed information [Sch09].

Virtual keyboards are typically smaller than personal computer keyboards, and therefore, it is difficult to type with 10 fingers. Because the keys themselves are smaller and harder to hit and provide no tactile feedback, the MobileApp must be tested to ensure that it allows easy error correction and can manage mistyped words without crashing.

Predictive technologies (i.e., autocompletion of partially typed words) are often used with virtual keyboards to help expedite user input. It is important to test the correctness of the word completions for the natural language chosen by the user, if the MobileApp is designed for a global market. It is also important to test the usability of any mechanism that allows the user to override a suggested completion.

Virtual keyboard testing is often conducted in the usability laboratory, but some should be conducted in the wild. If virtual keyboard tests uncover significant problems, the only alternative may be to ensure that the MobileApp can accept input from devices other than a virtual keyboard (e.g., a physical keyboard or voice input).

21.3.3 Voice Input and Recognition

Voice input has become an increasingly common method for providing input and commands in hands-busy, eyes-busy situations. Voice input may take several forms with different levels of programming complexity required to process each. Voicemail input occurs when a message is simply recorded for playback later. Discrete word recognition can be used to allow users to verbally select items from a menu with a small number of choices. Continuous speech recognition translates dictated speech into meaningful text strings. Each type of voice input has its own testing challenges.

According to Shneiderman [Shn09], all forms of voice input and processing are hindered by interference from noisy environments. Using voice commands to control a device impresses a greater cognitive load on the user, as compared to pointing to a screen object or pressing a key. The user must think of the correct word or words to get the MobileApp to perform the desired action. However, the breadth and accuracy of speech recognition systems are evolving rapidly, and it is likely the voice recognition will become the dominant form of communication in many MobileApps.

Testing the quality and reliability of voice input and recognition should take environmental conditions and individual voice variation into account. Errors will be made by users of the MobileApp and by the portions of the system processing the input. The MobileApp should be tested to ensure that bad input does not crash the MobileApp or the device. Large numbers of users and environments should be involved to be sure the MobileApp is working with an acceptable error rate. It is important to log errors to help developers improve the ability of the MobileApp to process speech input.

21.3.4 Alerts and Extraordinary Conditions

When a MobileApp runs in a real-time environment, there are factors that may impact its behavior. For example, a Wi-Fi signal may be lost, or an incoming text message, phone call, or calendar alert may be received while the user is working with the MobileApp.

These factors can disrupt the MobileApp user's work flow, yet most users opt to allow alerts and other interruptions as they work. A MobileApp test environment must be able to simulate these alerts and conditions. In addition, you should test the MobileApp's ability to handle alerts and conditions in a production environment on actual devices (Section 21.9).

Part of MobileApp testing should focus on the usability issues relating to alerts and pop-up messages. Testing should examine the clarity and context of alerts, the appropriateness of their location on the device display screen, and when foreign languages are involved, verification that the translation from one language to another is correct.

Many alerts and conditions may be triggered differently on various mobile devices or by network or context changes. Although many of the exception-handling processes can be simulated with a software test harness, you should not rely solely on testing in the development environment. This again emphasizes the importance of testing the MobileApp in the wild on actual devices.

Many computer-based systems must recover from faults and resume processing with little or no downtime. In some cases, a system must be fault tolerant; that is, processing faults must not cause overall system function to cease. In other cases, a system failure must be corrected within a specified period of time or severe economic damage will occur.

Recovery testing is a system test that forces the software to fail in a variety of ways and verifies that recovery is properly performed. If recovery is automatic (performed by the system itself), reinitialization, checkpointing mechanisms, data recovery, and restart are evaluated for correctness. If recovery requires human intervention, the mean time to repair (MTTR) is evaluated to determine whether it is within acceptable limits.